LAB04: CONDITIONAL EXPRESSIONS / FLOW CONTROL

CONTENT

Objective 2
CORE Guidelines & Naming conventions 3
naming conventions 3
Core guidelines 3
Exercises 4
Ex00: HANDS-ON Demo: enum class 4
Ex01: flow control basics 4
general 4
Logical operators 4
Switch and if statements 5
Scoped enumerations - ActorState 8
Using the struct Point2f 8
Ex02: Awaits Human Review 9
Ex03: **Hands-on demo:** sdl Events 10
hands-on demo + explanation 10
FYI: Polling events 10
EXERCISE: Handle the events 10
Ex04: Explosion 12
GENERAL 12
description 12
Ex05: Click Rectangle 13
GENERAL 13
generating the RECTANGLE IN A RANDOM SIZE AND POSITION 14
expected behavior 14
Check 14
PART 3: Bouncing line 14
Ex06: growing bars 15
PART 0: Create the first two bars on top 15
PART 1: Add bar that grows only every x steps 15
part 2: random color when clicking inside bar 16
Ex07: Coding Craftsmanship Challenge 16
Instructions 16

Wall of Fame 16
Submission instructions 17
Before zipping 17
Submit 17
References 17
Input/output manipulators - Operator Precedence 17
Leap year 17
Enumeration 17
Structs 17
ASCII table 18
Algorithm to draw circles and ellipses 18
SDL events 18
Event driven programming 18
SDL event 18
Keyboard button event 18
SDL Keycode Lookup Table 18
Mouse button event 18
Mouse motion event 18

OBJECTIVE

At the end of these exercises, you should be able to:

- Use the logical &&, || and ! operators
- Use the selection statements:
  - switch
  - if - else if – else
- Define and use enumerations
- Define and use new types using struct
- Use IO manipulators to change the precision and width of output
- Handling input events from the user:
  - keypress,
  - moving/clicking the mouse,
  - close the application.

We advise you to **make your own summary of topics** which are new to you.

CORE GUIDELINES & NAMING CONVENTIONS

NAMING CONVENTIONS

The **naming conventions** used in Prog1 and Prog2 promote a consistent coding style and should be followed, just as you will need to follow the conventions of any project, team, or company you end up with.

Category	Naming Convention / Pattern	Code Example
Local Variables	l**o**wer**C**amel**C**asing	`float width{};` `const float innerSize{ 10.f };`
Global Variables	**g_**UpperCamelCasing	`int g_NumberOfEnemies{};` `const float g_WindowWidth{ 500.0f };`

Boolean Identifiers	Verb indicating boolean state (Is..., Has…)	`bool isReady{};`
		`bool g_IsAlive{};`
		`bool IsKeyPressed();`

Functions / Methods	**U**pper**C**amel**C**asing (Verb-based)	`ClearBackground(); // return type void`
Types (Struct, Enum)	**U**pper**C**amel**C**asing	`struct Circle{…};` `enum class LightState{…};`
Enum *Values*	l**o**wer**C**amel**C**asing	`enum class Direction {none, leftUp, rightDown};`

CORE GUIDELINES

Always check the **PROG1: Naming Conventions & C++ Core Guidelines** document for a detailed explanation about the guidelines below.
(**New guidelines** introduced in this lab are indicated in **blue**)

- ✓ **F1;** Name meaningful operations
- ✓ **F3;** Keep functions short and simple
- ✓ **Enum.1;** Prefer enumerations over macros
- ✓ **Enum.3;** Prefer class enums over plain enums
- ✓ **Enum.5;** Don't use ALL_CAPS for enumerators
- ✓ **ES.5;** Keep scopes small
- ✓ **ES.10;** Declare one name (only) per declaration
- ✓ **ES.20;** Always initialize an object
- ✓ **ES.23;** Prefer the {} initializer syntax
- ✓ **ES.48;** Avoid casts if possible
- ✓ **ES.87;** Don't add redundant == or != to conditions
- ✓ **CON.1;** By default, make objects immutable
- ✓ **NL.1;** Don't say in comments what the code already says
- ✓ **NL.3;** Keep comments crisp
- ✓ **NL.4;** Maintain a consistent indentation style
- ✓ **NL.5;** Don't encode type information in names
- ✓ **NL.8;** Use a consistent naming style
- ✓ **NL.9;** Use ALL_CAPS only for macros
- ✓ **NL.20;** Don't place two statements on the same line
- ✓ **NR.1;** Don't force all declarations to the top

EXERCISES

Each lab you should **create a folder** with the correct name:

- **L03_ Name_FirstName**
  - Replace **Name** and **FirstName** by your personal last and first name
    - Avoid any special characters like ` or - by replacing them with an underscore _
  - Replace **L02** with the correct lab number each week
  - All projects from this lab should be created inside this folder

-- Remember from now on, we will use the `const` *qualifier proactively when creating variables that do not change value! --* ***C++ Coding Standards***

```
const int maxValue{ 100 }; //good if maxValue never changes
const int diameter { radius * 2 }; // good (even if radius is not const!)
```

EX00: HANDS-ON DEMO: ENUM CLASS

```
Horizontal direction: 0
Vertical direction: 1
Plane position after move: 30, 80
```

We start with a small demo about ***enum class*** *and compare them with* ***structs****. If you already feel familiar with both concepts in c++, you may continue with the next assignments.*

EX01: FLOW CONTROL BASICS

GENERAL

- Create a **PROG1 CONSOLE PROJECT** with above **EX number and title** (UpperCase) in this lab's folder
- Topics needed for this exercise: conditional expressions, flow control, enumerations

Make following exercises. Keep in mind that your code should be easy-to-read, e.g. **create a function** for each exercise. We defined following functions, notice we use upper camel casing for the function name, this is part of our naming standards:

```
void CheckLeapYear();
void ConvertSeconds();
```

Another advantage of using functions in this project is that you do not need to execute all the functions every time you launch the application, **just comment the ones that run fine**.

```
//CheckLeapYear();
ConvertSeconds();
```

LOGICAL OPERATORS

-- As you know by now, printing boolean values to the output window results in 0 for false, or 1 for true. By setting the boolalpha flag for our output, true or false will be printed instead! --

```
std::cout << std::boolalpha;
```

PRINT THE TRUTH TABLE

Create a function for this part where we will print the result of applying the logical operators to two bool variables. Don't forget to call it. Use this link to know how the Operator Precedence C++ is organized.

- Define 2 `bool` variables, one having the value *true* the other the value *false*.

- Print all possible combinations of both **variables** using operators below (**ignore** any **warnings** you get)
  - ✓ Logical **AND** operator **&&**

```
-- Print truth table --
Logical binary AND-operator (&&)
true && true is true
true && false is false
false && true is false
false && false is false
```

  - ✓ Logical **OR** operator **||**

```
Logical binary OR-operator (||)
true || true is true
true || false is true
false || true is true
false || false is false
```

  - ✓ Logical **NOT** operator **!**

```
Logical unary NOT-operator (!)
!true is false
!false is true
```

SWITCH AND IF STATEMENTS

COMPARE TWO NUMBERS

```
-- Compare 2 numbers --
First number is 8
Second number is 9
Second number is larger than first one
```

- Create a function that generates two random numbers in the inclusive interval [0, 10].
- Print both numbers followed by a message that indicates which one is larger. Cover **all** possibilities.
- Ensure that you don't always have the same sequence of random numbers.

CALCULATE PRICE

```
-- Calculate price --
Nr of ballpoints? 15
Total price is 60
```

*Example **input** of user in this screenshot: 15. Total price is calculated based on this.*

Create a function that asks the user the number of ballpoints they wish to order. Next, show the total price of the ballpoints based on this price table of the company:

Number	**Interval**	**Price a piece**
Less than 20	[0,19]	4,00 €
20 to 49	[20,49]	3,50 €
50 to and including 100	[50,100]	3,25 €
Over 100	[101,…	3,10 €

CONVERT DAY NUMBER

Create a function that shows information about a day based on the day number that was given by user input.

- Ask the user to enter a number ranging from 1 till 7. Let's assume that Monday is the first day of the week. Using an if-else structure, display a message telling the user whether the number corresponds with a "WEEKEND" day or "NO WEEKEND" day.

```
Day number [1, 7] ? 1
no weekend
```

```
Day number [1, 7] ? 6
weekend
```

```
Day number [1, 7] ? 7
weekend
```

- Extend the if-else structure by showing a more detailed message instead of "no weekend" for Monday to Friday, however still not mentioning the day itself.

```
Day number [1, 7] ? 1
Start of the week
```

```
Day number [1, 7] ? 2
Middle of the week
```

```
Day number [1, 7] ? 5
Nearly weekend
```

Day	Message
1	Start of the week
[2,4]	Middle of the week
5	Nearly weekend
[6,7]	Weekend
Other	Wrong day number

- Add a switch statement that displays the name of the day.

```
Day number [1, 7] ? 1
Monday
```

```
Day number [1, 7] ? 2
Tuesday
```

```
Day number [1, 7] ? 3
Wednesday
```

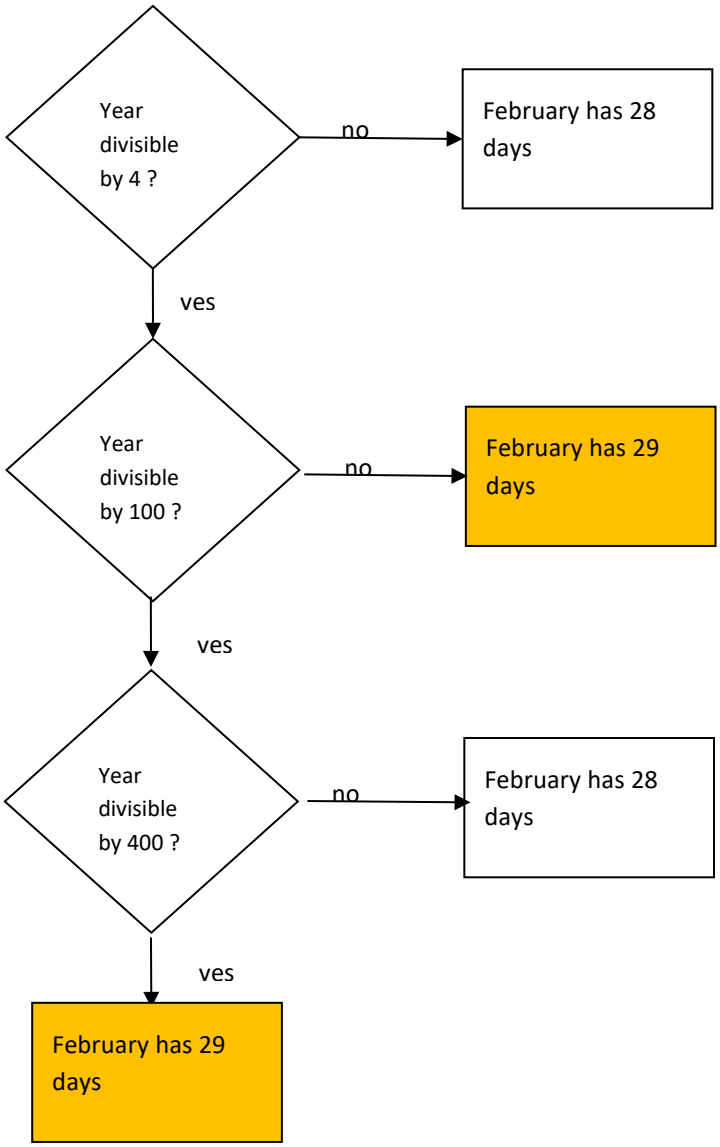
CHECK FOR LEAP YEAR

Feel free to comment out previous function calls so you do not have to enter the input for other exercises first!

Create a function that give the number of days in February based on the year that the user entered. This means we will have to determine if that year is a leap year.

```
-- Check leap year --
Year ? 1900
1900: 28
```

A **leap year** is a year containing one additional day in February (29 instead of 28 days). The **flow chart on the next page** shows how you can determine whether a year is a common or a leap year.



TEST EVERY POSSIBLE BRANCH!

Always test your code properly. You should test each possible branch in the selection statement.
By using next values all branches are tested:

Year	Number of days in February
2014	28
2012	29
2000	29
1900	28

CONVERT SECONDS: USING THE MODULO % OPERATOR

```
-- Convert seconds --
Number of seconds ?100000
1 days
3 hours
46 minutes
40 seconds
```

*This is an exercise to get you accustomed to using the **modulo operator %** . It is **not** practicing if statements.*

Create a function that asks the user for a number of seconds. Based on that, your program should show the number of days, hours, minutes and seconds that would result in. **Hint:** the modulo **%** operator can help here.

SCOPED ENUMERATIONS - ACTORSTATE

*-- We will create enumerations. Check the theory and your notes on this if necessary before you continue. If you need extra info, check the start demo + this url on **Scoped Enumerations**.*

Create a function where we will explore enumerations by creating one that keeps track of the actor state in a game (running, flying, falling)

- Define a scoped enumeration class `ActorState` with 3 possible values (enumerators): *running*, *flying* and *falling*.
- Define a variable of this new type and initialize it with one of the 3 enumerators.
- Test this variable:
  - Use a switch case
  - Print an appropriate message (**string literal**) for each of the possible enumerator values
  - Example when initialized with the *running* enumerator:

```
-- Define and use ActorState --
running
```

- Change the code to use a **random** value for this state variable.
  - It is not possible to randomize an enumeration directly.
  - Hint on how to proceed:
    - Generate a random number within the right interval. **Min** boundary: cast the first enumerator to an int. **Max** boundary: cast the last enumerator to an int.
    - Cast this random integer number to `ActorState` type
    - use it to initialize the `ActorState` variable
- Test again with the switch case you wrote before.
  Example:

```
-- Define and use ActorState --
flying
```

USING THE STRUCT POINT2F

Last week we introduced the Point2f struct. Let's go deeper into the possibilities to initialize it.
Create a function for it and **copy this struct inside the function**:

```
struct Point2f
{
    float x;
    float y;
};
```

Define 3 variables of this new type in different ways:

- p1: without initialization.
- p2: with uniform default initialization.
- p3: with uniform initialization using literals e.g. 26 and 25

Print the Point2f values on the console, you can access their elements x and y using the dot operator. This leads to a problem for p1, because it was not initialized. Don't print that one, comment that code. Remember that you should always initialize local variables.

Then change their values like this:

- p1: assigning the value 20 to x and the value 3 to y
- p2: assigning the value Point2f {30,40}
- p3: applying the ++ and -- operator on x and y

And print them again, now p1 is printed without any problems because a value was assigned to it.

```
-- Define and use Point2f --
Values after definition
p2: [0, 0]
p3: [26, 25]
Values after changing the members
p1: [20, 3]
p2: [30, 40]
p3: [27, 24]
```

EX02: AWAITS HUMAN REVIEW

Unzip the **given cmake project** with the same name as the title above.

You have been given **a piece of AI-generated C++ code**. Your task is to give it a code review and identify mistakes, bad practices, guideline violations, and logic errors.

> ***!! THIS WEEK, A LOT OF LOGIC ERRORS (= PROGRAM RUNS, BUT WRONG OUTPUT IN CONSOLE) ARE IN THE CODE! FIND, MARK AND CORRECT ALL OF THEM.***

Your goal

- Use the Core guidelines & naming conventions section at the start of this document as reference.
  - The new ones introduced in this lab are indicated in **blue**.
- For every guideline violation, **mention the corresponding guideline code** (e.g. ES.23, NL.5).
- **Write your findings as comments** above or next to the relevant line(s) of code.
- Always **explain in short** why the code is problematic, not just what is wrong.
- For each issue you find, comment out original code and **write the corrected version underneath it**.

IMPORTANT

IF YOU ARE PLANNING TO ASK AN AI TO REVIEW THE AI-GENERATED CODE FOR YOU, PLEASE SKIP THIS ASSIGNMENT ENTIRELY. YOU'LL SAVE:

- *YOUR TIME,*
- *OUR TIME,*
- *A FEW SERVERS,*
- *AND POSSIBLY AN ENTIRE FOREST WORTH OF ELECTRICITY.*

More importantly, you'll miss the actual learning goal: developing the ability of critical thinking. In a few weeks, you won't be graded on your ability to ask an AI for answers. You'll be graded on your ability to review code yourself. This will be done in a **closed-book** environment, without AI, notes, or online resources.

EX03: **HANDS-ON DEMO:** SDL EVENTS

- Create a new **PROG1 GAME PROJECT** with above **EX number and title** (UpperCase) in this lab's folder
- Make sure you set build dependencies and title

HANDS-ON DEMO + EXPLANATION

-- Time to add event handling to our projects! This part will be demonstrated by the teacher.
More information about event driven programming can also be found here and README.md--

FYI: RUN FUNCTION

Let's have a better look at our **Prog1Engine** project:

- In the `main.cpp` file, you see that in the `main` function, the `Run` function is called.
- The definition of this `Run` function can be found in the `core.cpp` file.
- Check it out: the `Run` function contains a loop consisting of:
  1. A call to the `Start` function, used to load game resources such as images and sounds (later).
  2. **Code that handles input (events) from the user**, such as:
     - pressing a **key** while the window has the focus,
     - moving/clicking the **mouse** on the window,
     - closing the window…
     - these functions are in the `main.cpp` file.
  3. Code that calculates the **elapsed time** between two draw calls
  4. A call to the `update` function that is in the `main,cpp`.
  5. A call to the `Draw` function (`main.cpp`)
  6. Swapping front and back buffer (`SDL_GL_SwapWindow`)

FYI: POLLING EVENTS

SDL stores events in an event queue. With the **`SDL_PollEvent`** function, you get information about the next event in this event queue. And it also removes that event from the queue.

This function results in an `int` type, that has the value…

- … 1 when there was an event available in the queue,
- .. 0 when none was available.

Information about an event is wrapped in an **`SDL_Event`** type variable that is passed as an argument to this function. The function puts information about the polled event in this argument.

When there are no longer events in the queue, then we execute the `Draw` operation.

For more information see SDL event

EXERCISE: HANDLE THE EVENTS

PART 1: TRACE SOME COMMON EVENTS

Let's handle some events using a `switch` statement.
Show an appropriate message on the console for every event function in `Game.cpp`

- For example: print "a key has been pressed" in the corresponding event
- Do the same thing for other key and mouse events

PART 2: GET EXTRA EVENT INFORMATION

`SDL_Event` contains next to the event type several pieces of extra information.

- Show extra information on the console as indicated in next table and screenshot.
- In particular:
  - show a message when the left/right arrow key is released,
  - and when the left, middle and right mouse button went down.

In the functions, there is commented code that can be quite useful. To find out how other keys can be checked, click `SDLK_LEFT` and press F12. A new window opens showing a sdl header file.

Event type (e.type)	Extra information	In which event part
SDL_KEYUP SDL_KEYDOWN Keyboard button event SDL Keycode Lookup Table	Which virtual key	e.key.keysym.sym
SDL_MOUSEMOTION Mouse motion event	Mouse position	e.motion.x , motion.y
SDL_MOUSEBUTTONDOWN	Mouse position	e.button.x , button.y
SDL_MOUSEBUTTONUP Mouse button event	Which button	e.button.button

```
SDL_MOUSEBUTTONDOWN
  [393, 111]
  Left mouse button went down

SDL_MOUSEBUTTONUP

SDL_MOUSEBUTTONDOWN
  [393, 111]
  Middle mouse button went down

SDL_MOUSEBUTTONUP

SDL_MOUSEBUTTONDOWN
  [435, 34]
  Right mouse button went down

SDL_MOUSEBUTTONUP

SDL_KEYDOWN

SDL_KEYUP
  Left arrow is released

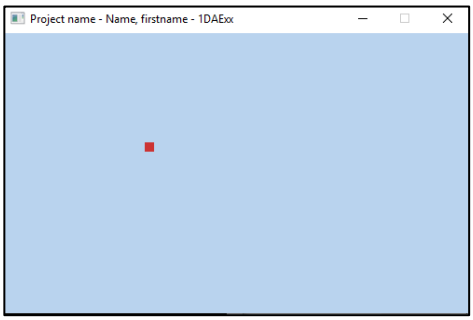
SDL_KEYDOWN

SDL_KEYUP
  Right arrow is released
```

PART 3: FILL A LITTLE SQUARE ON THE MOUSE POSITION

Let's visualize the position of the mouse in the screen, by making a square follow the current mouse position.

- We need to **keep track of the mouse position**
  - Define a `Point2f` type variable to store this. Should it be local or global?
    Hint: we also need it in `Draw` to draw the square
  - In which **event** should you update this variable?
    When unsure, try a few mouse events to see the difference!



- We need to **draw** the square in this position
  - Make sure you create a function for it and call it from `Draw`
    Note: *we do not draw in an event!*
  - Draw a red square of size 10, for example

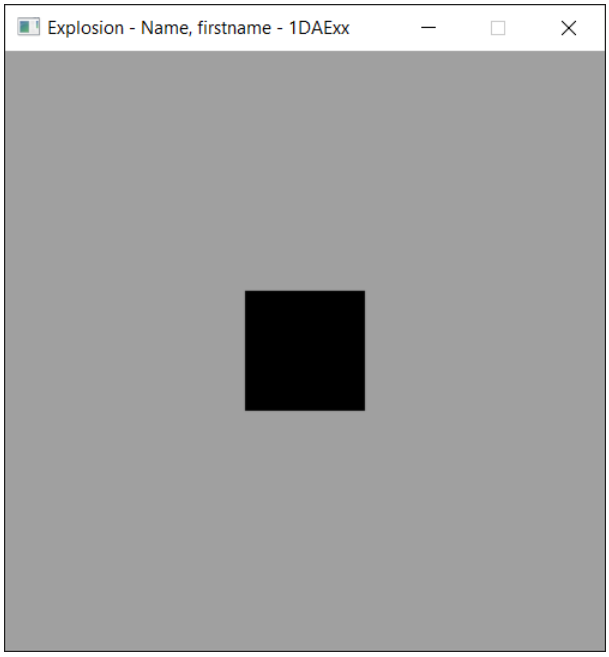
EX04: EXPLOSION

GENERAL

- Create a new **PROG1 GAME PROJECT** with above **EX number and title** (UpperCase) in this lab's folder
- Make sure you set build dependencies and title
- Make the window a square by choosing the same width and height.

DESCRIPTION

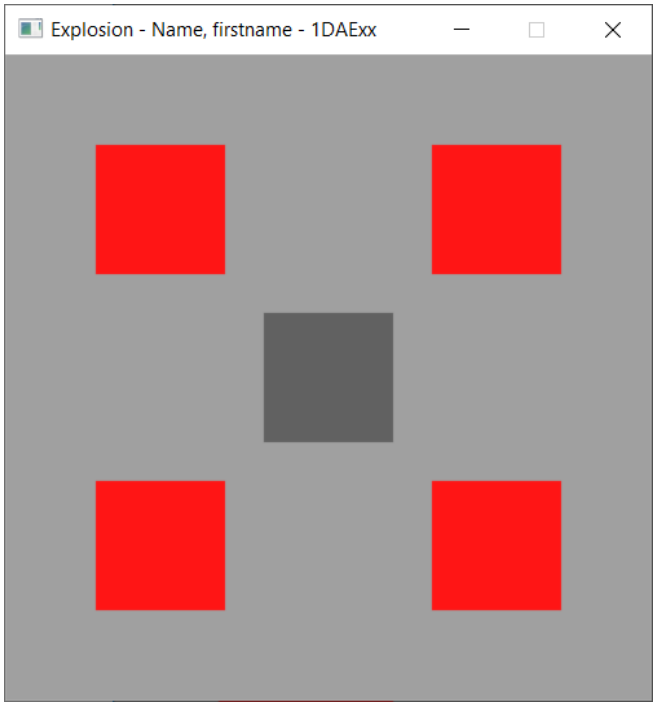
In this exercise a bomb – a filled black square - is located in the center of the window.



When clicking with the left mouse button ***inside*** **the center rectangle** (`SDL_BUTTON_LEFT`): it becomes partly transparent and 4 red filled rectangles start moving towards the corners of the window. Once they reach the sides, they reset to the center.

⇨ Store the **mouse position** in a **global** + find the **game event** that would respond to a **mouse click**.

Once all rectangles leave the (square) window, the whole thing resets as if a new bomb is planted.



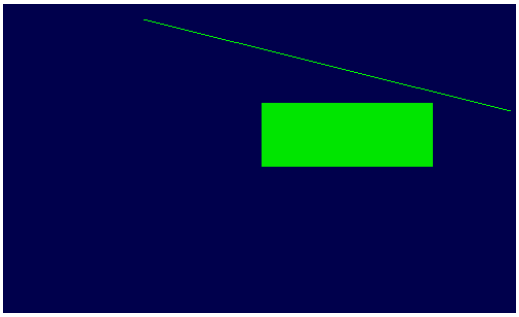
Make your code readable, create **functions**, e.g.:

- A function that **starts** the explosion, another that **resets** it.
  - Think carefully which game functions would call these helper functions.
- A function that **changes** the location of the explosion squares on the window.
  - Remember **not** to call this method from the Draw() game function.
- A function that **draws** the bomb and the explosion squares.

Hint: since all squares start from the center, there is no need for you to keep the position of each square separately. Recalculate the positions before drawing instead.

EX05: CLICK RECTANGLE

GENERAL



- Create a new **PROG1 GAME PROJECT** called **Ex05_ClickRectangle** in this lab's folder
- You can find a screencast of this

Hints while going through all instructions below:

- Use **functions** when appropriate! Think carefully about when to call them.
- Use boolean variables to handle the various states in the game.
- Remember to store the mouse position in a global variable so it can be used elsewhere.

Think of using global variables that are:

- changed in the **event** functions;
- checked/used/changed in the **update** and **draw** functions.

GENERATING THE RECTANGLE IN A RANDOM SIZE AND POSITION

Generate and draw a rectangle in a random size, using a random position:

- It is only random once; not every frame!
- Its minimum size is 60 x 40 pixels.
- It never spawns closer than 50 pixels from the window borders. (Hint: re-use part of your code from a previous lab.)
- Determine the rectangle's colour while drawing; do not store it in a separate global variable.

EXPECTED BEHAVIOR

- Initially, there is a gray rectangle in a random position that does not move.
- When you click inside the rectangle, it turns green (does not affect movement).
    - If you click it again, it goes back to gray.
    - At this point (only), its size and position also regenerate (random).
    - **Hint:** do **not** store the color in a global variable!
- Pressing the R key can toggle the movement of the rectangle:
    - The rectangle starts moving when the R key is pressed.
    - When the R key is pressed while it is already moving, it stops.
    - It always moves towards the right side of the window.
    - Each pixel that leaves the window on the right appears again on the left edge of the window.
        - Think carefully about what trick you could use to obtain this effect.
    - Clicking the rectangle still has the same effect as described above and is therefore independent of the movement.

CHECK

- Test all behavior as described above.
- A moving rectangle can also be clicked and keeps showing expected behavior.
- Make sure the part of the rectangle that appears on the left when the other exceeds the right side of the window can also be clicked.
- A final check for your code quality: change the colors of the rectangle, for example:
    - Blue instead of gray when not moving
    - Red instead of green when moving.
    - *If all goes well, you should only have to change 2 lines of code (1 for each color) in a **drawing** helper function. If not, remember to store the **state** rather than the color as a global variable!*

PART 3: BOUNCING LINE

Look at the bouncing line in the screencast. It is separate from the rectangle. Therefore, start by creating functions in your header file for each behaviour of the lines (drawing, moving, bouncing, etc.).

- Consider creating a **struct** for each line end:
    - Position (x and y)
    - Horizontal speed
    - Vertical speed
    - ⇨ Next, create 2 instances for this struct; one for the **start** and another for the **end** of the line.
- Both line ends **start** in a random position *inside* the window. Their initial **speed** in all directions is 2.
- Both line ends **bounce** off the edges of the window.
- Look for “bouncing line screen saver” screencast - you implement just one line.

EX06: GROWING BARS



Create a **Prog1 Game Project** called **EX06_GrowingBars**.

Check the screencast in this week's source material and try to recreate it following instructions below:

- Draw and update 3 bars. They all have different sizes and fill colors.
- Every frame, their fill increases with a certain step, where one fills faster than the other.
- The last bar only updates every 50 frames.

PART 0: CREATE THE FIRST TWO BARS ON TOP

- **Create a struct** to keep track of everything you need to remember about each bar (color, size,..)
- Create two instances of this type to represent the yellow and red bar in the screenshot / -cast.
- Draw and update these two bars every frame. Once they are full, their fill resets back to 0.
- They update with a different speed.

PART 1: ADD BAR THAT GROWS ONLY EVERY X STEPS

- Add an extra bar to it (green for example) in the same way you did for the other two
- Check the screencast: the green bar should **fill in 8 steps, but grows only every 50 frames**.
  ➔ (Only) if you do not know how to get started, read these hints:
  - We need to create a variable that keeps track of how many frames already passed *(how many times has the update loop been executed?)*
  - We need another that knows how many frames it should wait (const)
  - Only once it has actually waited enough frames, the fill width of the green bar is increased
  - Make sure it starts waiting again after this *(what must be reset?)*
  - Once the green bar is full, just like the other bars, it must reset back to 0
  - Last but not least: how do you make sure the bar is filled in exactly 8 steps?
- If the user hits the **'r' key**, all bars should be **reset** to fill 0.

PART 2: RANDOM COLOR WHEN CLICKING INSIDE BAR

Every time the user clicks inside the lowest bar, it should get a random color

- Get the mouse position if the user clicks
- Check if it is inside the boundaries of the new bar
- If so, generate a **random color:**
  - Random value between 0.00 and 1.00 for red,
  - Random value between 0.00 and 1.00 for green,
  - Random value between 0.00 and 1.00 for blue
  - *(Remember how to generate a random floating point value with 2 decimals?)*

EX07: CODING CRAFTSMANSHIP CHALLENGE

Level up your game development powers! This is where creativity meets code, and it bridges the gap between the traditional step-by-step written lab assignments and a much deeper understanding of the material.

In this weekly exercise you're tasked with inventing an exercise that merges all c++ programming and game development skills you obtained so far in this course into a (small!) unique project. Let your imagination run wild and work on a programming project YOU love!

Do not underestimate the value of this exercise; experimenting is by far the best way to master programming!

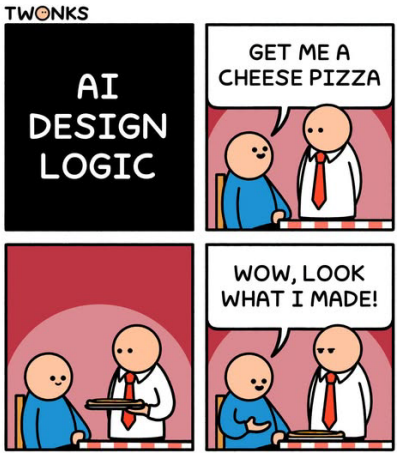
Create either a **console** **or** a **Prog1 Game Project** called CodingCraftmanship04 in the 1DAEXX_01_NAME_FIRSTNAME folder.

Use as much of the new topics as possible. This is where the fun really starts: now that we handled events and flow control, we can easily create small games!

DO NOT SKIP THIS EXERCISE; EXPERIMENTING WITH CODE IN A SELF-MADE PROJECT IS BY FAR THE BEST WAY TO MASTER PROGRAMMING!

INSTRUCTIONS

Create a **Prog1 Game Project** called **CodingCraft03** that demonstrates **animations**.



Create a **small original application** that demonstrates the concepts we've covered so far.

- ✓ Create **your own** original project.
- ✓ **Only** use **concepts that have been covered** in labs already.
- ✓ Write code you can explain yourself.
- x **No** AI-generated code.
- x **No** copied solutions from the internet.
- x **No** copying another student's work.

For example, this week: draw your favorite logo or character using basic shapes or find inspiration by searching for daily minimal in Google images.

WALL OF FAME

Every week, some projects will earn a place on the lab's Wall of Fame.

A project does not need to be complex to deserve a spot. Original ideas, clever concepts, fun themes, polished outputs, and well-written code are just as valuable. Projects are shown **anonymously** so you can choose whether you reveal yourself as the creator or not.

The **goal** is to **learn, experiment, and become a better programmer**. The Wall of Fame only works if every project genuinely reflects your own work. **Let your imagination run wild and work on a programming project YOU love!**

SUBMISSION INSTRUCTIONS

You must submit **two files**:

- A **zip file** containing your **complete lab folder** of this week with all exercises.
- **CodingCraft03.zip** containing only your Coding Craftsmanship Challenge project.

BEFORE ZIPPING

For every project:

- **Close** Visual Studio.
- Delete the **out** folder.
- Delete the hidden **.vs** folder.

SUBMIT

- Upload **both files** with the correct assignment before the start of next week's first lab.
- Make sure to complete the **quiz**.
- *These steps are required to unlock the next lab assignment!*

⚠️ Your submission may be discussed during a code review. Make sure you understand and can explain your own code.

REFERENCES

INPUT/OUTPUT MANIPULATORS - OPERATOR PRECEDENCE

http://en.cppreference.com/w/cpp/language/operator_precedence

LEAP YEAR

https://en.wikipedia.org/wiki/Leap_year#Algorithm

ENUMERATION

https://www.youtube.com/watch?v=Pxvvr5FAWxg
http://www.learncpp.com/cpp-tutorial/4-5a-enum-classes/

STRUCTS

http://www.learncpp.com/cpp-tutorial/47-structs/

ASCII TABLE

http://www.learncpp.com/cpp-tutorial/27-chars/

ALGORITHM TO DRAW CIRCLES AND ELLIPSES

http://www.mathopenref.com/coordcirclealgorithm.html

SDL EVENTS

EVENT DRIVEN PROGRAMMING

http://lazyfoo.net/tutorials/SDL/03_event_driven_programming/index.php

SDL EVENT

https://wiki.libsdl.org/SDL_Event

KEYBOARD BUTTON EVENT

https://wiki.libsdl.org/SDL_KeyboardEvent

SDL KEYCODE LOOKUP TABLE

https://wiki.libsdl.org/SDLKeycodeLookup

MOUSE BUTTON EVENT

https://wiki.libsdl.org/SDL_MouseButtonEvent

MOUSE MOTION EVENT

https://wiki.libsdl.org/SDL_MouseMotionEvent